# Elchai Website Redesign — Full Codebase Audit

**Codebase audit**
2026-05-25

# CONTENTS

# Elchai Website Redesign — Full Codebase Audit

**Repository:** https://github.com/Elchaigroup/ElchaiWebsiteRedesignSMARTPANTS_PRESERVED_426 **Local copy audited:** `/Users/elchai/Projects/elchai-redesign` at commit `c1f158c` (8 commits, `main`) **Stack:** Next.js 15.5 (App Router, Turbopack) · React 19 · TypeScript 5.9 (strict) · Tailwind v4 · Three.js · Framer Motion · Lenis · Radix · Resend · Zod 4 **Audit date:** 2026-05-25 **Method:** Four parallel specialist passes (architecture, security, performance, supply chain) plus direct reads of the API surface, build config, and a full `next build` to capture bundle weights.

---

## 1. Executive Summary

This is a serious, type-disciplined Next.js marketing site that is roughly 95% feature-complete. The architecture is conventional, the type-safety floor is unusually high, and there are no Critical security findings. The site builds cleanly (68 of 68 pages statically prerendered in 8.6s) and there is one dominant performance pothole that, fixed alone, takes the site from "competent" to "fast."

**The verdict at a glance:**

| Dimension | Grade | One-line reason |
|---|---|---|
| Architecture | A- | Principled server/client split, data-driven service pages, one tiny i18n smell |
| Type safety | A+ | Strict on, zero `eslint-disable`, zero `console.log`, 5 careful casts in the whole tree |
| Security | B+ | No Critical issues, no secret leaks; missing rate limits, security headers, constant-time compare |
| Performance | C+ | Build is clean and SSG is comprehensive, but every page ships ~215kB of Three.js eagerly |
| Production readiness | B | CI green, no deploy job, no Sentry/observability, no `global-error.tsx`, no health endpoint |
| Supply chain | B+ | 2 moderate npm vulns (easy fix), Playwright in prod deps, 2 dead Spline deps, no Dependabot |

**The 5 things to fix before launch (in this order):**

1. **Actually dynamic-import `BackgroundScene`.** The `.tsx` and `.impl.tsx` files are byte-identical 4,470-LOC twins — the wrapper claimed in the handoff doc was never wired. Replace `BackgroundScene.tsx` with a `next/dynamic({ ssr: false })` wrapper around `.impl.tsx`. **This single change drops `/[slug]` First Load JS from 962 kB to ~370 kB** across all 48 service pages, the homepage, and every other route that imports it (13 call sites).
2. **Rate-limit `/api/lead`** plus add Turnstile or hCaptcha. Today an attacker can flood the inbox, burn Resend quota, and thrash WordPress with one curl loop. Single most exploitable issue.
3. **Add security headers in `next.config.ts`** (HSTS, X-Frame-Options, Referrer-Policy, X-Content-Type-Options, Permissions-Policy). CSP later with nonces.
4. **Skip Three.js on mobile and low-power devices.** Add `if (innerWidth < 768 || navigator.deviceMemory <= 4)` short-circuit at `BackgroundScene.impl.tsx:~4392`. Today a 3-year-old Android does the same WebGL work as a 4K desktop.
5. **Add observability** (Sentry or equivalent) and an `app/global-error.tsx`. The site has `app/error.tsx`, `app/not-found.tsx`, and `app/loading.tsx`, but a root-layout throw still drops to a white screen. Launching without monitoring is launching blind.

---

# 2. Architecture and Code Quality

## 2.1 STRUCTURE

Conventional Next.js App Router. `src/app/` holds page-per-folder routes (`about-us`, `contact`, `portfolios`, `blog/[slug]`, `[slug]`, `api/lead`, `api/revalidate`, `og`, `sitemap.xml`, `robots.txt`, `llms-full.txt`). Server components are the default; per-page client subtrees are split into `*Body.tsx` files (e.g., `src/app/about-us/page.tsx` plus `AboutUsBody.tsx`). The split is disciplined and idiomatic.

The dynamic catch-all at `src/app/[slug]/page.tsx` looks up `getServiceDetailContent(slug)`, renders the universal `<ServiceDetail/>` template when found, and falls back to a “Coming soon” stub otherwise. About 47 service pages exist as data entries, not files. `generateStaticParams` at `:15-17` lists all known slugs, so 48 service pages prerender at build time.

## 2.2 COMPONENT LAYOUT

`src/components/` is split into:

- **`primitives/`** — 14 small atoms (`Reveal`, `Parallax`, `PrimaryCTA`, `GhostCTA`, `EyebrowPill`, etc.)
- **`sections/`** — 30 page sections
- **`ui/`** — 8 Radix / 21st.dev wrappers

71 `"use client"` files in total (mostly `sections/` plus every `*Body.tsx`). Server components own data and metadata; client components own interaction. No accidental client bleed.

### 2.3 STATE

Almost stateless. Only **one** `createContext` in the entire codebase (`src/lib/i18n.tsx:56` for `LangCtx`). Everything else is local `useState` plus URL-hash-driven modals (`ModalsHost.tsx:5`). No Redux, no Zustand. For a content site, this is the right choice.

### 2.4 TYPE SAFETY — EXCEPTIONAL

- `tsconfig.json:7` has `"strict": true`.
- **Zero** **`eslint-disable`** comments in `src/`.
- **Zero** **`console.log`** in source — only legitimate `console.error` / `console.warn` in error handlers.
- ESLint config (`eslint.config.mjs:23`) enforces `@typescript-eslint/no-explicit-any: error` and the rules pass.
- Only 5 `as unknown as` casts in the whole tree; 3 are defensible cross-locale module casts with explanatory comments.

This is one of the cleanest type-safety surfaces seen in a marketing-site codebase.

### 2.5 STYLING

Tailwind v4 with `@theme` tokens in `src/app/globals.css:13–64`. Brand palette is a single source of truth: `--color-brand-sky #24E5FF`, plus layered surface tokens, cool-grey text scale, and 10-step monochrome. Fonts are wired to next/font CSS variables so the loaded woff2 is actually consumed. `src/lib/tokens.ts` mirrors the same palette in TS. `cn.ts` is the standard `clsx + tailwind-merge` helper.

Concrete tokenization with no inline-color sprawl — coherent.

## 2.6 I18N (EN / AR / IT)

Client-side only. `src/lib/i18n.tsx:58–90` provides `<LangProvider>` with cookie plus localStorage persistence and flips `<html lang/dir>` for RTL Arabic. UI strings live in `src/locales/{en,ar,it}.json` (small, 86 lines each). Long-form content uses parallel modules: `content.ts/.it.ts/.ar.ts` plus per-vertical service modules.

This is **not** Next's native `app/[locale]` i18n; URLs stay single-locale. The static HTML shipped to search engines is always English. AR/IT only appear post-hydration after a client-side mutation. Crawlers will never index AR/IT content under this model. The hreflang declarations in `src/lib/seo.ts:80–87` all point to the same path, which is technically correct for a single-URL site but provides no per-locale ranking benefit.

**Content scale:** `services/ai.{ts,it.ts,ar.ts}` is ~3,480 LOC each; `blockchain` is 2,182 × 3. Total ~18,000 LOC of typed content modules. All three locales' dictionaries are bundled into every page's client JS (static imports in `i18n.tsx:17–19`), adding ~30-100 kB to every route. A Zod schema (already a dep) checked at build would catch EN/AR/IT shape drift cheaply.

## 2.7 CODE SMELLS

- **The big one:** **`BackgroundScene.tsx`** **and** **`BackgroundScene.impl.tsx`**. Both 4,470 LOC, byte-identical (confirmed by `wc -l` and `head -5`). The `.impl` suffix was meant to enable a dynamic-import wrapper that does not exist — `grep -rn "dynamic(" src` returns zero hits. Result: Three.js is fully bundled into every page's client chunk. See §4 for the bundle impact.
- **`src/lib/_legacy/service-detail-content.legacy.ts`** — 8,624 LOC quarantined in `tsconfig.json:22` exclude and `eslint.config.mjs:13`. Properly fenced but should be deleted, not archived in-tree.
- **`@splinetool/react-spline`** **plus** **`@splinetool/runtime`** in `package.json:20–21` — **zero imports in** **`src/`**. Dead deps, ~6.6 MB on disk.
- **`README.md`** **is one line.** Onboarding outside the author's head depends entirely on `HANDOFF.md`.
- **TODO / FIXME markers in src/: zero.** Either remarkable discipline or aggressive cleanup.
- **`HANDOFF.md`** **punch list** still has 4 unaddressed items (lines 194-209): Case Studies vertical-space trim, Stats numbers visual alignment, `Parallax` position-static console warning, and a likely-wrong `+47` WhatsApp default dial code for a Dubai HQ.

### 2.8 TESTS

Single Playwright spec: `e2e/smoke.spec.ts` (129 LOC). Covers 8 representative routes returning 200 with no real console errors, two SEO redirects, sitemap/robots shape, `/api/lead` happy path plus honeypot plus bot-time guard, and `#consultation` hash modal opening.

**No unit tests.** No Vitest/Jest. For a content-heavy marketing site with very little business logic this is defensible — the data files are statically typed and would benefit more from schema validation than unit tests. The Playwright suite is small but well-targeted at the things that actually break.

### 2.9 DOCS

- `HANDOFF.md` **(328 lines)** — exemplary. Snapshot tables, locked decisions, owner-relationship notes. Real onboarding artifact.
- `design-system.md` **(801 lines)** — locked visual spec. The codebase follows it (token names match).
- `CHANGES.md` **(115 lines)** — copy plus override log.
- `README.md` — one line. The single real documentation gap.

---

## 3. Security

**Verdict: no Critical findings.** No live secrets in source or git history, no XSS sinks fed by user input, no exposed admin routes, no SSRF on user-controlled URLs, no client-leaked credentials. Severity inflation deliberately avoided.

### 3.1 HIGH SEVERITY

**H1. `/api/lead` has no rate limiting, no CAPTCHA, no IP throttling**

- `src/app/api/lead/route.ts:45-115`
- Defenses are only a Zod schema, a honeypot field (`website`), and a 1.2s time-to-submit guard (`ttsMs < 1200`). No `@upstash/ratelimit`, no IP check, no WAF rule in repo.
- **Exploit:** scripted POST `{"source":"consultation","name":"x","email":"x@x.io","ttsMs":1500}` in a loop. Each request fans out to (a) `LEAD_WEBHOOK_URL`, (b) WordPress REST, (c) Resend email. Outcomes: inbox flood at `info@elchaigroup.com`, Resend quota burn (paid tier), WordPress DB bloat, possible Resend reputation damage causing real leads to land in spam.

- **Fix:** Per-IP rate limit (5 requests / 10 min / IP) via `@upstash/ratelimit`. Add Cloudflare Turnstile or hCaptcha on the consultation form. Honeypot plus 1.2s gate stops only naive bots.

#### H2. Lead handler trusts `lead.email` as `replyTo` — operator phishing surface

- `src/lib/email/sendLead.ts:112` — `replyTo: lead.email`
- Zod's `.email().max(200)` validates format and Resend's SDK does its own header sanitization, but the value lands in a real RFC-5322 header. The operator clicking "Reply" then sends a response to an attacker-controlled address.
- **Exploit:** attacker submits a lead with `reply@lookalike-attacker.com`; operator replies casually with internal context that becomes spear-phishing material.
- **Fix:** drop `replyTo` entirely. Lead emails are internal notifications; staff should reply via CRM, not directly to the lead's email-of-record through the website pipeline.

#### H3. Outbound webhook URLs read from env — operator misconfig risk

- `src/app/api/lead/route.ts:26-43`, `src/lib/wp/lead.ts:29-31`
- `LEAD_WEBHOOK_URL`, `WP_LEAD_ENDPOINT`, `WP_BASE_URL` are all `process.env`-sourced and fetched server-side with `AbortSignal.timeout(5000)`. Not user-controlled, so this is **not** classic SSRF.
- The risk: if an operator pastes an internal URL into env (`http://localhost:9200`, `http://169.254.169.254/...` AWS metadata), the server hits it on every lead. Lower severity because operator-only.
- **Fix:** enforce `https://` scheme and public-host validation in `forward()` and `forwardLeadToWp()`. Reject RFC1918 / link-local.

### 3.2 MEDIUM SEVERITY

#### M1. No security headers in `next.config.ts`

- No `async headers()` block. No `middleware.ts` anywhere. `poweredByHeader: false` is set (good), nothing else is.
- Site can be iframed on attacker domain (clickjacking the consultation/WhatsApp CTAs). No CSP means a future XSS becomes catastrophic — no defense in depth. Full Referer leaks on outbound clicks.

- **Fix:** add `headers()` returning:
  - `Strict-Transport-Security: max-age=63072000; includeSubDomains; preload`
  - `X-Frame-Options: SAMEORIGIN` (or CSP `frame-ancestors 'self'`)
  - `Referrer-Policy: strict-origin-when-cross-origin`
  - `X-Content-Type-Options: nosniff`
  - `Permissions-Policy: camera=(), microphone=(), geolocation=()`
  - Eventually a CSP — tricky because of the inline JSON-LD script tags in `layout.tsx:303-310`, `FAQ.tsx:44-47`, `ServiceDetail.tsx:269-274`. Use nonces or pragmatic `'unsafe-inline'` for `script-src` initially.

### M2. Revalidate route uses non-constant-time string comparison

- `src/app/api/revalidate/route.ts:47-50` — `if (provided !== secret)`
- Theoretically timing-vulnerable; in practice network jitter dominates so risk is low.
- **Fix:** `crypto.timingSafeEqual(Buffer.from(provided ?? ""), Buffer.from(secret))` after a length check.

### M3. Revalidate accepts arbitrary `tag` / `path` arrays — cache thrash DoS if secret leaks

- `src/app/api/revalidate/route.ts:67-76`. Once authed, attacker can POST `{"path":["/","/a","/b",...]}` with 1,000 entries in a tight loop. No length cap, no per-secret rate limit.
- **Fix:** cap `tags.length + paths.length <= 50`. Validate each tag matches `^wp:(posts|post:[\w-]{1,200})$`. Validate each path matches `^/[\w\-/]{0,200}$`. Add IP rate-limit.

### M4. WordPress Basic-Auth on every `wpFetch` — no scheme check on `WP_BASE_URL`

- `src/lib/wp/client.ts:25-35, 86-88`. If operator misconfigures `WP_BASE_URL=http://...`, the Basic Auth header (trivially base64-reversible to `user:password`) goes over plaintext on every request.
- **Fix:** reject `baseUrl` that does not start with `https://` in `getWpConfig()`.

## 3.3 LOW SEVERITY

- **L1.** `/api/lead` returns Zod issues to client (`route.ts:55-58`), telling bot authors the field names. Strip `issues` from the response; log server-side.
- **L2.** Honeypot / `ttsMs` bot rejects silently return `200`. Intentional but unlogged — add a counter so the operator can monitor false-positive rate on real users.
- **L3.** Spline plus YouTube iframes without `sandbox` attribute or CSP `frame-src` allowlist.

- **L4.** `html-react-parser` in `PrivacyPolicyBody.tsx:131` and `TermsOfServiceBody.tsx:131` parses inline-defined HTML literals. Safe today, but a future swap to WP/CMS source would create stored-XSS. Add a comment to that effect.

### 3.4 WHAT IS CLEAN — EXPLICITLY

- **Secrets hygiene.** Grep for `sk_live_`, `AKIA*`, `AIzaSy*`, `ghp_`, `Bearer` across `src/` and `scripts/` returned no matches. `.gitignore` correctly excludes `.env`, `.env*.local`. `git log --all --diff-filter=A` shows only `.env.example` (all values blank) was ever committed. `.mcp.json` contains only a public WordPress.com MCP URL.
- **Unsafe HTML injection** — 3 usages of React's raw-HTML escape hatch, all safe. Every payload is `JSON.stringify(<hardcoded object>)` inside a `type="application/ld+json"` script tag (not executable JS). No user input flows in. No `eval`, no Function-constructor calls, no direct `.innerHTML` assignment in client code.
- **Auth / authz** — N/A. No admin routes, no sessions, no JWT, no cookies set by the app. Pure marketing site.
- **CORS** — default (none). Same-origin only.
- **Client-side data exposure** — clean. Zero `NEXT_PUBLIC_*` env references. No PII in localStorage. All env access is server-side.
- **Git history** — 8 commits total. Pickaxe on `sk_`, `SECRET`, `PASSWORD` only surfaced `.env.example` and env-var documentation. No leaked credentials.

---

## 4. Performance and Production Readiness

Based on a real `next build` (8.6s compile, 68/68 static pages, zero warnings, zero TS errors) plus direct read of `next.config.ts`, the bundle output, and the rendered app structure.

### 4.1 BUILD OUTPUT — THE HEADLINE

| Route | First Load JS | Verdict |
|---|---|---|
| `/` (homepage) | 383 kB | Heavy but acceptable |
| `/[slug]` (service pages × 48) | **962 kB** | Red flag |
| `/blog/[slug]` | ~370 kB | Fine |
| `/api/*`, `/sitemap.xml`, `/robots.txt` | 102 kB shared only | Fine |

Largest single chunk: `.next/static/chunks/912-3f8f171109283601.js` = **215 kB compressed** (~700+ kB uncompressed). Almost certainly the Three.js bundle.

### 4.2 THE DOMINANT PROBLEM: `BACKGROUNDSCENE` IS NOT ACTUALLY DYNAMIC-IMPORTED

`BackgroundScene.tsx` (4,470 LOC, `"use client"`) imports `three` and six `three/examples/jsm/...` modules **eagerly at module top** (lines 17-24). It is imported as a static `import { BackgroundScene }` in **13 separate route files**:

```
src/app/page.tsx:22
src/app/[slug]/page.tsx:7
src/app/contact/page.tsx:6
src/app/about-us/page.tsx:6
src/app/interns/page.tsx:6
src/app/portfolios/page.tsx:6
src/app/privacy-policy/page.tsx:6
src/app/terms-of-service/page.tsx:6
src/app/blog/[slug]/page.tsx:8
src/app/blog-list/page.tsx:7
src/app/live-demo/page.tsx:9
src/app/case-study/page.tsx:6
src/app/not-found.tsx:6
```

The handoff doc claims a “dynamic-import wrapper” was complete. It is not — `BackgroundScene.tsx` and `BackgroundScene.impl.tsx` are byte-identical 4,470-LOC twins, and `grep -rn "dynamic(" src` returns zero hits. So Three.js ships fully bundled to every visitor on every page.

The runtime mount strategy at `BackgroundScene.tsx:4372-4448` is excellent — paints a static radial gradient first, waits for `window.load` then `requestIdleCallback` with a 2.5s safety fallback, respects `prefers-reduced-motion`, caps `MAX_PIXEL_RATIO: 2.0`, has WebGL `try/catch` with gradient fallback, cleans up listeners cleanly — but that defers **execution**, not **download**. The browser still parses 215 kB of Three.js on every navigation.

**The fix is small.** Replace `BackgroundScene.tsx` with:

```
"use client";
import dynamic from "next/dynamic";
export const BackgroundScene = dynamic(
  () => import("./BackgroundScene.impl").then(m => m.BackgroundScene),
  { ssr: false, loading: () => null }
);
```

This alone drops `/[slug]` First Load from 962 kB to ~370 kB across all 48 service pages and the homepage. Effort: ~1 hour.

### 4.3 NO MOBILE / LOW-POWER SHORT-CIRCUIT

`BackgroundScene.tsx:4388-4393` checks `prefers-reduced-motion` but does not check device capability. On a low-end Android, the same 4,470-LOC WebGL scene runs as on a 4K desktop — ~2.5 MB of decoded textures plus the scene graph. Add:

```
if (window.innerWidth < 768
    || (navigator as any).deviceMemory <= 4
    || navigator.hardwareConcurrency <= 4) {
  applyStaticFallback();
  return;
}
```

Effort: ~30 min. Mobile B2B traffic is real and this is where jank surfaces.

### 4.4 WHAT IS CONFIGURED WELL

- `next.config.ts:13` — `transpilePackages: ["three"]` for clean Three.js examples/jsm transpilation.
- `next.config.ts:15` — `optimizePackageImports: ["lucide-react", "framer-motion"]` so icon imports tree-shake instead of pulling the whole library.
- `next.config.ts:18` — `formats: ["image/avif", "image/webp"]` for next/image.
- `next.config.ts:19-24` — `remotePatterns` tight; only YouTube plus own domain. No `**` wildcards.
- `next.config.ts:10` — `poweredByHeader: false` (one less fingerprint).
- `package.json:11` — `analyze` script wires `@next/bundle-analyzer`.
- 26 source files import `next/image`. **Zero raw image tags** in `src/`.
- next/font self-hosts Montserrat plus Inter with `display: swap`. Fontshare Satoshi and Google Fonts Geist/JetBrains Mono are deferred to a post-hydration `FontLoader.tsx` (deliberate FOUT trade for 600-1200ms LCP improvement, well documented in the file).
- 68/68 pages prerender statically. No `export const dynamic = "force-dynamic"` leaks anywhere.
- `app/error.tsx`, `app/not-found.tsx`, and `app/loading.tsx` all exist.
- Custom `Cache-Control` set on the four route handlers (`robots.txt`, `sitemap.xml`, `llms-full.txt` → 1 hr; `og` → 1 year immutable).

## 4.5 OTHER CONCRETE FIXES

- **`public/elchai/hero-key-bg.png` is 1.5 MB.** Convert to WebP/AVIF (~150 kB) or delete if the Three.js hero superseded it.
- **`@splinetool/react-spline` plus `@splinetool/runtime`** in `package.json:20-21` — zero imports. Delete both. (~6.6 MB freed on install.)
- **`playwright` is in `dependencies` not `devDependencies`** (`package.json:28`). Move it. (~16 MB freed on prod install.)
- **`@next/bundle-analyzer ^16.2.6` paired with `next ^15.1.0`** (`package.json:17` vs `:27`) — major-version mismatch. Pick one.
- **No `app/global-error.tsx`.** If the root layout itself throws, users get a blank page.
- **No observability.** No Sentry, no PostHog, no Plausible, no GA. You cannot see real-user errors or Web Vitals in production.
- **No `/api/health` endpoint.** Uptime monitors have to ping `/` or `/sitemap.xml`.
- **No `vercel.json`, `Dockerfile`, `fly.toml`, `netlify.toml`, `render.yaml`.** Deployment target is undeclared in the repo.

## 4.6 SEO — BEST-IN-CLASS

- Per-page `metadata` via `pageMetadata()` in `src/lib/seo.ts:55-106`; used on every static route.
- `generateMetadata` on `/[slug]` and `/blog/[slug]`.
- Canonical handling: layout deliberately leaves global canonical blank (`layout.tsx:56-68` comment) so per-page canonicals work.
- Stub service pages get `robots: { index: false, follow: true }` AND are filtered out of the sitemap (`sitemap.xml/route.ts:61-68`). Excellent.
- JSON-LD: Organization plus WebSite (with SearchAction) plus ProfessionalService inlined in `layout.tsx:128-289`. BreadcrumbList helper in `seo.ts:113-150`. FAQPage conditional in `ServiceDetail.tsx`. AggregateRating was correctly **removed** to avoid Google manual-action risk.
- `robots.txt` explicitly allows GPTBot, ClaudeBot, PerplexityBot, OAI-SearchBot — good answer-engine posture.
- `sitemap.xml` merges static plus service plus WP blog posts with a stable `lastmod`.
- `/llms.txt`, `/llms-full.txt`, and `<link rel="alternate">` in head — all present.
- Dynamic OG image at `/og` (`src/app/og/route.tsx`) with `cache-control: immutable, max-age=31536000`.

This is well above the marketing-site median.

---

# 5. Dependencies and Supply Chain

## 5.1 VULNERABILITIES

`npm audit` headline: **2 moderate, 0 high, 0 critical.**

| Package | Severity | Issue | Fix |
|---|---|---|---|
| `postcss <8.5.10` (transitive, via `next`) | Moderate (CVSS 6.1) | XSS via unescaped `</style>` in stringify output (GHSA-qx2v-qp2m-jg93) | `npm update postcss` (8.5.15 wanted, already in range) |
| `next` | Moderate (only reflected because of postcss) | Same root issue | Same fix |

The `fixAvailable.version: 9.3.3` for `next` in the audit JSON is an `npm audit` quirk — `next` itself is current; updating the nested `postcss` resolves both lines. Run `npm audit fix` and re-run.

## 5.2 OUTDATED

| Package | Current | Latest | Notes |
|---|---|---|---|
| `next` | 15.5.18 | 16.2.6 | One major behind. Not security-driven. |
| `framer-motion` | 11.18.2 | 12.40.0 | One major behind. |
| `three` | 0.160.1 | 0.184.0 | Caret on `0.x` pins to `0.160.x` only — no minor updates flow in. |
| `@types/three` | 0.160.0 | 0.184.1 | Matches above. |
| `lucide-react` | 0.470.0 | 1.16.0 | Crossed `1.0`. Same caret-on-0.x trap. |
| `typescript` | 5.9.3 | 6.0.3 | Major. |
| `eslint` | 9.39.4 | 10.4.0 | Major. |
| `eslint-config-next` | 15.5.18 | 16.2.6 | Pair with `next` upgrade. |
| `@next/bundle-analyzer` | 16.2.6 declared | — | **Mismatched: `^16.2.6` analyzer paired with `next ^15.1.0`** in `package.json:17` vs `:27`. Unsupported pairing. |

## 5.3 FOOTPRINT

- `node_modules` total: **531 MB**.
- Largest dirs: `next/` 153 MB, `@next/` 124 MB (SWC binaries), `lucide-react/` 36 MB, `three/` 32 MB, `typescript/` 23 MB (dev), `@img/` 16 MB (sharp).
- Single lockfile: `package-lock.json` only. Clean.
- Dead deps: `@splinetool/react-spline` plus `@splinetool/runtime` (6.6 MB combined, zero imports).

## 5.4 PINNING AND HYGIENE

- **No `engines` field, no `packageManager` field, no `.nvmrc`, no `.node-version`.** Node version is unpinned. CI uses `actions/setup-node@v4` so CI is fine; local devs and Docker builds have no guardrail.
- **No `.github/dependabot.yml`, no `renovate.json`.** No auto-update strategy.
- **`playwright ^1.59.1` is in `dependencies` not `devDependencies`** (`package.json:28`). `@playwright/test` is correctly in devDeps. Move it.

- No git URLs, no `file:` paths, no tarball deps. Clean on supply-chain registry mixing.
- Postinstall scripts in `node_modules`: only `sharp` (legit, for next/image) and `unrs-resolver` (ESLint resolver). No suspicious payloads.

### 5.5 GIT HISTORY

- 8 commits total on `main`.
- `.env*` never committed (only `.env.example` template, all values blank).
- Pickaxe for `sk_live`, `RESEND_API_KEY=re_`, `WP_APP_PASSWORD=`, `PASSWORD=` returns nothing.
- Commit author email is malformed (concatenated four times: `elchaiworld@gmail.comel-chaiworld@gmail.com…`). Worth fixing in git config for credibility.

---

## 6. CI and Deploy

`.github/workflows/ci.yml` (sole workflow, 47 lines): on push to `main` and every PR runs `install → typecheck → lint → build → Playwright smoke`. Uploads HTML report on failure. 15-minute timeout. Concurrency cancellation. **No deploy job.**

`package.json` scripts: `dev` (Turbopack on :3000), `build`, `typecheck`, `lint`, `analyze`, `test:e2e`. Standard.

Gaps:

- No deploy workflow → manual or platform-managed; not documented in repo.
- CI actions are major-version-pinned (`@v4`), not SHA-pinned. Standard practice but SHA-pinning is a minor supply-chain hardening opportunity.
- No status badge in README.
- No deploy-target config files in repo (`vercel.json` / `Dockerfile` / `fly.toml` / `netlify.toml` / `render.yaml`).

---

## 7. Final Recommendations — prioritized

### BEFORE LAUNCH (DO THESE)

1. **Replace `BackgroundScene.tsx` with a `next/dynamic` wrapper around `.impl.tsx`.** ~1 hr. Drops 600 kB off every page.
2. **Rate-limit `/api/lead`** plus add Turnstile / hCaptcha. (H1)

3. **Add security headers** in `next.config.ts` — HSTS, X-Frame-Options, Referrer-Policy, X-Content-Type-Options, Permissions-Policy. (M1)
4. **Drop** `replyTo: lead.email` in `src/lib/email/sendLead.ts:112`. (H2)
5. **Mobile / low-power short-circuit** in `BackgroundScene.impl.tsx`.
6. **Add** `app/global-error.tsx` and wire Sentry (or equivalent).
7. `npm audit fix` to clear the 2 moderate `postcss` vulns.
8. **Move** `playwright` **to devDependencies** (`package.json:28`); delete dead `@splinetool/*` deps (`package.json:20-21`).
9. **Fix** `@next/bundle-analyzer` **16.x vs** `next` **15.x mismatch** (`package.json:17` vs `:27`).
10. **Convert or delete** `public/elchai/hero-key-bg.png` (1.5 MB).
11. **Constant-time secret compare plus payload cap** in `/api/revalidate`. (M2 + M3)
12. **Add a real** `README.md` pointing to `HANDOFF.md` and `design-system.md`.
13. **Pin Node version** — add `engines.node` to `package.json` and a `.nvmrc`.

## FIRST WEEK POST-LAUNCH

14. Delete the duplicate of `BackgroundScene.tsx` / `BackgroundScene.impl.tsx` (whichever ends up dead after step 1).
15. Delete `src/lib/_legacy/` (it is git-history-only material).
16. Add `/api/healthz` for uptime monitoring.
17. Run `npm run analyze` and verify the chunk size drop after fix #1.
18. Wire `dependabot.yml` for weekly minor plus patch PRs.
19. SHA-pin GitHub Actions in `.github/workflows/ci.yml`.

## QUARTER-1 BACKLOG

20. Migrate to native Next i18n (`app/[locale]/...`) so AR/IT get prerendered, canonical URLs, and no EN-FOUC.
21. Add Zod schemas at build time over locale content modules to catch EN/AR/IT shape drift.
22. Reject HTTP `WP_BASE_URL` and RFC1918 hosts in `wpFetch`.
23. Add unit tests for the lead pipeline if Resend / WordPress integration grows beyond best-effort.
24. Split locale dictionaries per locale via dynamic import.

---

## 8. Bottom Line

This codebase is in better shape than most marketing-site rebuilds. The type discipline is unusually high, the architecture decisions are coherent, the SEO surface is exhaustive, and there are no live security catastrophes. The build is clean and 68 of 68 pages prerender statically.

The dominant risk between “looks great in dev” and “fast and safe on the open internet” is one specific code-smell: the Three.js scene is statically imported on every route, shipping 600 kB of WebGL bundles to visitors who may never reach the canvas-mount idle callback. Fix that, rate-limit the lead endpoint, add security headers, and wire monitoring — and this is genuinely ready to ship.

Estimated effort for the full “before launch” punch list: ~1 working day.